UCLM

PROYECTO DISIA

Desarrollo e Integración de Servicios de Inteligencia Artificial (DISIA)

José Antonio de la Torre las Heras

Escuela Superior de Informática

Escuela Superior de Ingeniería Informática

Proyecto DISIA

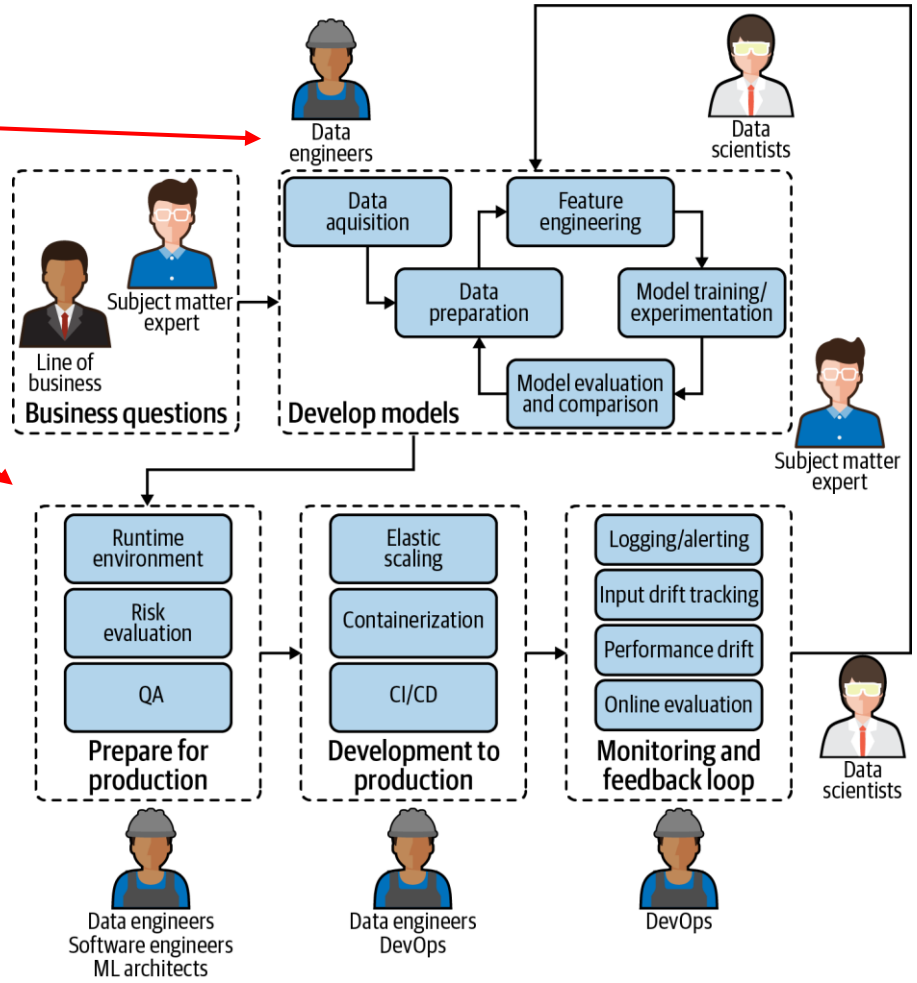
- Hito 4:
  - Despliegue y paso a producción del modelo
    - Selección de las tecnologías para el despliegue del modelo
    - Del notebook al sistema
    - Análisis de las necesidades del sistema (software)
  - Fecha límite: 05/05/2026

Requisitos de la entrega

- Tener separado el entrenamiento e inferencia en dos contenedores diferentes
- Implementar una API para la inferencia
- Despliegue con Docker-compose entrenamiento e inferencia
- Documentación
- Bonus:
  - Uso de kubernetes
  - Uso de minio para almacenamiento de modelos
  - Uso de mlflow para experimentación

Proyecto DISIA: Hito 4

- Hasta ahora habéis:
  - Hito 1: diseñado y validado un sistema IA
  - Hito 2: analizado, seleccionado y explorado los datos
  - Hito 3: selección de modelo, diseño de experimentos, validación de hipótesis y resultados



Proyecto DISIA: Hito 4

- ¿Qué haréis en el hito 4?
  - Convertir los experimentos en sistemas funcionales
  - Desplegar vuestro servicio IA en una primera infraestructura funcional
  - Analizar las necesidades de vuestro sistema
    - ¿Requiere de GPU?
    - ¿Es intensivo en CPU?
    - ¿Necesitamos almacenar mucho vídeo para luego hacer la inferencia?

Proyecto DISIA: Hito 4

- **Ejemplo**: California Housing Dataset
- Pasar de notebook a:
  - Estructura independiente: entrenamiento, training
  - Contenerización
  - Despliegue

Ejemplo

- El notebook está dividido en:
  - Ingesta de datos
  - Descripción y análisis de datos
  - Ingeniería de características
  - Limpieza de datos
  - Evaluación de modelos
  - Inferencia

Ejemplo II

- Lo vamos a dividir en:
  - src/
    - data_ingestion.py: encargado de cargar los datos. Simulamos una fuente de datos externa y los hace accesible al pipeline, en este caso un csv.
    - features.py: los procesos para la ingeniería de características.
    - train.py: carga los datos, los preprocesa y entrena el modelo. Como resultado genera un modelo exportable.
    - infer.py: carga el modelo, proporciona el método predict.
- Reglas:
  - Las rutas siempre serán configurables por comandos o variables de entorno

Ejemplo III

```python
parser = argparse.ArgumentParser(description="Pipeline de Feature Engineering")
parser.add_argument("--input", type=str, default="data/raw/california_housing.csv", help="Ruta del CSV en crudo")
parser.add_argument("--output", type=str, default="data/processed/california_housing_features.csv", help="Ruta destino del CSV procesado")
parser.add_argument("--limits", type=str, default="data/artifacts/outlier_limits.json", help="Ruta para guardar el artefacto JSON")
args = parser.parse_args()

input_path = Path(args.input)
output_path = Path(args.output)
limits_path = Path(args.limits)
```

Ejemplo IV: Contenerización

- Diferenciar las dos fases en al menos dos contenedores
  - Entrenamiento
  - Inferencia
- Esto permitirá luego aplicar el feedback loop
- Ficheros:
  - Dockerfile.train
  - Dockerfile.infer
  - Docker-compose.yml

Ejemplo V: preguntas abiertas

- ¿Qué ocurre si los datos cambian?
- ¿Qué ocurre si el modelo cambia?
- ¿Cómo accedo a las inferencias desde fuera?

Ejemplo VI: API

- Acceso desde el exterior al proceso de inferencia
- ¿Cómo son las cargas de trabajo?
  - Batch
  - Streaming
- ¿Qué tipo de clientes vamos a tener?
  - Móviles
  - Dispositivos IoT
  - Televisores
- Tecnologías
  - Flask
  - Fastapi
  - Mlrun
  - Kserve

Ejemplo VII: API

```python
import os
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field
from infer_mlflow import HousingPredictor

# Instanciamos la aplicación FastAPI
app = FastAPI(
    title="API de Predicción de Precios (California Housing)",
    description="API MLOps para inferencia en tiempo real",
    version="1.0.0"
)

# Cargamos el predictor en el contexto global (se ejecuta al arrancar el servidor)
predictor = HousingPredictor(model_name="California_Housing_Prod", stage="None")

# Definimos el contrato de entrada (Esquema de datos)
class HousingInput(BaseModel):
    MedInc: float = Field(..., description="Ingreso medio en decenas de miles", json_schema_extra={"example": 8.3252})
    HouseAge: float = Field(..., description="Edad de la casa en años", json_schema_extra={"example": 41.0})
    AveRooms: float = Field(..., description="Promedio de habitaciones", json_schema_extra={"example": 6.9841})
    AveBedrms: float = Field(..., description="Promedio de dormitorios", json_schema_extra={"example": 1.0238})
    Population: float = Field(..., description="Población del bloque", json_schema_extra={"example": 322.0})
    AveOccup: float = Field(..., description="Ocupación promedio", json_schema_extra={"example": 2.5555})
    Latitude: float = Field(..., description="Latitud", json_schema_extra={"example": 37.88})
    Longitude: float = Field(..., description="Longitud", json_schema_extra={"example": -122.23})
```

Ejemplo VIII: API

```python
@app.post("/predict")
def predict_price(data: HousingInput):
    """
    Endpoint principal para predecir el precio.
    """
    try:
        # data.dict() convierte el objeto Pydantic a un diccionario normal
        raw_prediction = predictor.predict(data.dict())

        # Multiplicamos por 100,000 por la naturaleza del dataset original
        final_price = raw_prediction * 100000

        return {
            "status": "success",
            "model_version": "1.0",
            "estimated_price_usd": round(final_price, 2)
        }
    except Exception as e:
        # Si algo falla en el modelo, devolvemos un error HTTP 400 limpio
        raise HTTPException(status_code=400, detail=f"Error en la predicción: {str(e)}")
```

Preguntas abiertas

- ¿Una API para cada modelo?
- ¿Qué ocurre si cambian las características?
- ¿Un cliente por API?

Soluciones estándar

- **Open Inference Protocol**: usado por kserve, soporta http, grpc...
  - Unifica los parámetros para la inferencia
  - Proporciona endpoints para:
    - Salud
    - Metadatos del modelo
  - Permite su uso mediante
    - REST
    - GRPC
  - Implementado por:
    - NVIDIA Triton Inference Server protocol
    - Seldon Core v2 inference protocol
    - OpenVINO REST/gRPC API, AMD Inference Server y TorchServe Inference API

```
{
  "id": "string", // optional
  "parameters": {}, // optional
  "inputs": [
    {
      "name": "string",
      "shape": [1, 2, 3],
      "datatype": "FP32",
      "parameters": {}, // optional
      "data": [1.0, 2.0, 3.0]
    }
  ],
  "outputs": [ // optional
    {
      "name": "string",
      "parameters": {} // optional
    }
  ]
}
```

Soluciones de facto

- Open AI Responses API
- HuggingFace